Revision 42-45 · Tuning Memory & Long Prompts

Interactive revision of lesson 42, lesson 43, lesson 44 and lesson 45: each question as asked, the answer, where I slipped, and the one-line takeaway.

Q1 · "We can only serve 3 users?" PARTIAL

Covers: 19, 42

> **QUESTION**
>
> Llama 3.1 8B on one H100 with vLLM defaults. The startup log says "Maximum concurrency for 131,072 tokens per request: 3.43x", and the PM panics that only 3 users fit per GPU. Rebuild 3.43 from the 54.8 GiB KV pool, and say how many 1,800-token chats actually fit.

How I solved it

1. KV per token = 2 (K and V) × 32 layers × 8 KV heads × 128 dims × 2 bytes = 131,072 bytes = **128 KiB**.
2. Pool in tokens = 54.8 GiB ÷ 128 KiB = 54.8 × 2^{20} KiB ÷ 2^{7} KiB = 54.8 × 8,192 ≈ **449,000 tokens**.
3. The log divides the pool by `max_model_len`: 449,000 ÷ 131,072 ≈ **3.43**. That's how many **maximum-length** requests fit, a worst case.
4. Real chats only take the blocks they use: 1,800 × 128 KiB = 230,400 KiB ≈ 225 MiB each, and 449,000 ÷ 1,800 ≈ **249 chats**.

The PM is wrong: about 250 chats fit at once. 3.43 only says that three 131k-token requests would fill the cache. Setting `--max-model-len 8192` changes the log to 54.8x without changing the pool.

> **RE-DRILL**
>
> Do the GiB → tokens step in powers of two without help: GiB × 2^{20} ÷ 128 = GiB × 8,192.

Q2 · Size max_num_seqs from real shapes PARTIAL

Covers: 41, 42

> **QUESTION**
>
> To stop preemptions, a teammate sets max_num_seqs = pool ÷ average request length = 449,000 ÷ 1,200 ≈ 374. The p95 request length is 2,400 tokens. What's wrong, and what value would you pick?

How I solved it

1. Sizing for the average is wrong: half the requests are longer, so 374 requests growing to their real lengths overflow the pool and trigger preemptions.
2. Size for a high percentile instead: 449,000 ÷ 2,400 ≈ **187** requests at p95 length.
3. Round **down** and keep a margin for variation: use about **160**.

`max_num_seqs` counts requests, not tokens, and too high is the dangerous direction: the vLLM default of 1,024 admits about 4× what can finish together (~250 chats), which leads to a preemption spiral.

> **RE-DRILL**
>
> Before picking a direction to round, ask which side of the limit causes the failure.

Q3 · Raise the utilization cap PARTIAL

Covers: 18, 42

> **QUESTION**
>
> You raise gpu_memory_utilization from 0.92 to 0.95 on a card where vLLM sees 79.1 GiB. How many extra KV tokens and extra 1,800-token chats does that buy? Someone suggests going straight to 0.99: what could go wrong, and when would you find out?

How I solved it

1. Extra memory = 0.03 × 79.1 GiB ≈ **2.4 GiB** (every 1% ≈ 0.79 GiB).
2. Extra tokens = 2.4 × 8,192 ≈ **19,400 tokens** (every 1% ≈ 6,500 tokens).
3. Extra chats = 19,400 ÷ 1,800 ≈ **10 chats**.
4. Risk of 0.99: the strip above the cap must hold everything vLLM doesn't allocate itself (other processes, fragmentation, NCCL buffers, spikes, and CUDA graphs on older versions). If it doesn't fit, you get **CUDA out of memory at runtime under load**, not at startup.

vLLM's auto-tuning script starts at 0.98 and steps down until the server starts cleanly.

> **RE-DRILL**
>
> When asked "what could go wrong", always add "and when would you see it".

Q4 · fp8 KV cache or 8-bit weights? MISSED

Covers: 18, 42

> **QUESTION**
>
> Same server (54.8 GiB pool, 128 KiB per token, 15 GiB of weights). You can flip one flag: fp8 KV cache, or 8-bit weights (15 → 7.5 GiB). Which adds more KV capacity for the 8B model, and does that change for 70B on two H100s?

How I solved it

1. **fp8 KV** halves the bytes per token (128 → 64 KiB), so the same pool holds **twice the tokens**: 449k → ~900k, **+449k**.
2. **8-bit weights** don't change bytes per token; they free 7.5 GiB that joins the pool: 7.5 × 8,192 ≈ **+61k tokens** (+14%).
3. For 8B, **fp8 KV wins ~7×**: the pool is much bigger than the weights.

70B, TP = 2 on 2× H100	Pool	Tokens (320 KiB each)
bf16 weights (≈ 131 GiB of the 146 GiB cap, ~6 GiB overheads)	≈ 8 GiB	≈ 26k
+ fp8 KV (160 KiB each)	≈ 8 GiB	≈ 52k (**+26k**)
+ 8-bit weights (frees ≈ 66 GiB)	≈ 74 GiB	≈ 240k (**+215k**)

1. For 70B, **8-bit weights win ~8×**: the weights dwarf the pool, so freeing them matters more than shrinking each token.

Both change the numbers the model computes with, so measure quality before shipping either.

> **RE-DRILL**
>
> For any "which knob" question, write the budget (cap = weights + overheads + pool) first and see which term the knob touches.

Q5 · "40 ÷ 8 = 5 GPUs" MISSED

Covers: 21, 43

> **QUESTION**
>
> The busiest minute averages 40 req/s. One H100 holds the SLO up to 8 req/s. The manager says 40 ÷ 8 = 5 GPUs. With a 20% burst margin and surviving the loss of one replica, how many GPUs do you really need?

How I solved it

1. Burst margin: usable rate per GPU = 8 × 0.8 = **6.4 req/s**.
2. Failure: after one of N replicas dies, the N − 1 survivors still carry all 40 req/s at ≤ 6.4 each, so N − 1 ≥ 40 ÷ 6.4 = 6.25.
3. Round up: N − 1 = 7, so **N = 8 GPUs**.

4. Check: normally $40 \div 8 = 5$ req/s each (62% of the benchmark). One dies: $40 \div 7 \approx 5.7 \leq 6.4$. With 7 GPUs, a failure gives $40 \div 6 \approx 6.7 > 6.4$, which fails.

8 GPUs, not 5: planning at about 60% of the benchmark number is what keeps the promise on a bad day. N + 2 (9 GPUs) is for services that must survive two losses at once.

> **RE-DRILL**
>
> Redo it for N + 2: survivors are $N - 2 \geq 6.25$, so 9 GPUs.

Q6 · The fastest config loses PARTIAL

Covers: 35, 43

> **QUESTION**
>
> A sweep gives three configs: A (max_num_seqs 256, 8,192 tokens per step) raw 11 req/s, SLO capacity 7; B (128, 2,048) raw 9.5, SLO 8; C (64, 2,048) raw 7, SLO 6. The team wants A because it's fastest. Which do you pick, and which target does A break first, and C?

Answer

- Pick **B**: it holds both targets up to the highest rate. Raw throughput keeps rising past the SLO edge, so the fastest config isn't the best one.
- **A breaks TPOT first:** up to 256 requests and 8,192 tokens in one step make every step slower, and a long prefill in that step stalls everyone who is decoding.
- **C breaks TTFT first:** only 64 requests run at once, so request 65 isn't preempted; it **waits in the queue** before it is ever admitted, and the wait counts toward its first token.

> **RE-DRILL**
>
> For each knob, ask "who pays: the users already running, or the ones waiting to get in?"

Q7 · Let more requests in? PARTIAL

Covers: 36, 41, 43

> **QUESTION**
>
> Traffic goes beyond plan and requests are waiting. A teammate raises max_num_seqs from the tested 128 to 512 to "let more in". What happens to users already streaming, and what should protect the operating point instead?

Answer

- Every step now carries up to 4× the requests, so **TPOT climbs for everyone** already streaming, and once the KV pool runs out, running requests get **preempted** and recomputed.

- **Keep max_num_seqs at the tested value:** overload then waits in the queue instead of slowing everyone who is already running.
- **A queue timeout:** an overloaded server sheds work it can't serve in time instead of piling it up.
- **Autoscale below the edge, not at it:** new GPUs take minutes to warm up.
- **Write it down** (config, SLO capacity, planned rate, margins) and rerun when the model, engine or traffic shape changes.

> **RE-DRILL**
>
> Separate "make capacity bigger" (lesson 42) from "keep load inside capacity" (lesson 43).

Q8 · How long does everyone freeze? PARTIAL

Covers: 04, 15, 16, 44

> **QUESTION**
>
> Thirty users stream from Llama 3.1 8B (16 GB in 16-bit) on an H100. One pastes an 8,000-token document, prefilled in one step. With 989 TFLOPS and 3.35 TB/s, how long do the streams freeze, and how many times a normal decode step is that?

How I solved it

1. Prefill is **compute-bound**: time = FLOPs ÷ FLOPS. FLOPs = 8,000 tokens × 2 × 8 × 10^9 = 1.28 × 10^{14} FLOP.
2. Freeze ≈ 1.28 × 10^{14} ÷ 9.89 × 10^{14} ≈ **0.13 s = 130 ms** (peak math, before attention; attention adds ~17 ms).
3. Decode is **memory-bound**: time = bytes ÷ bandwidth = 16 × 10^9 ÷ 3.35 × 10^{12} ≈ **4.8 ms**.
4. Ratio ≈ 130 ÷ 4.8 ≈ **27×**, in line with Sarathi-Serve's measured 28×.

> **RE-DRILL**
>
> Before dividing, say the units out loud: FLOP ÷ FLOP/s, or bytes ÷ bytes/s.

Q9 · "p99 didn't move, so no interference" PARTIAL

Covers: 22, 44

> **QUESTION**
>
> The repro runs 32 chats for 40 s with ~10 ms decode steps and one 8k prompt injected every 10 s. A teammate sees p99 inter-token latency unchanged from baseline and concludes there's no interference. Why might p99 miss it, and what would you report instead?

How I solved it

1. Gaps per user ≈ 40 s ÷ 10 ms ≈ **4,000** (about 3,940 once the spikes' own time is taken out).
2. Spikes per user = one per injection = **4**, so 4 ÷ 3,940 ≈ **0.1%** of gaps.
3. p99 looks at the slowest **1%**, i.e. ~40 gaps per user: 36 of them are normal steps, so p99 sits at baseline.
4. Report what can see 0.1%: the **max**, **p99.9**, the **stall count**, or the full **histogram** (the spikes form a separate bucket near 150 ms). Or inject more often (every 1 s pushes spikes above 1%), and line the spikes up with injector timestamps as proof.

TPOT is worse still: it averages each request's gaps, so 4 slow gaps out of thousands disappear.

> **RE-DRILL**
>
> Before trusting a percentile, compute "bad events ÷ total samples" and compare it with the tail share.

Q10 · The largest chunk inside a 20 ms budget PARTIAL

Covers: 16, 44, 45

> **QUESTION**
>
> Chunked prefill on the same server (8B, H100, 32 users decoding), with a 20 ms inter-token budget. Step time ≈ the slower of reading the weights (4.8 ms) or the math (tokens × 16 GFLOP ÷ 989 TFLOPS). What's the largest chunk that keeps every step inside 20 ms, and what would you round it to?

How I solved it

1. Math per token = $16 \times 10^9 \div 9.89 \times 10^{14} \approx$ **16 μs**.
2. Tokens that fit in 20 ms = 20 ms ÷ 16 μs = **1,250** per step.
3. The 32 decode tokens go in first: chunk ≤ 1,250 − 32 ≈ **1,218**.
4. Round **down** to a multiple of 128: **1,152** (1,280 would go over).
5. Lower edge for comparison: the weight read is 4.8 ms ÷ 16 μs ≈ 300 tokens of math. Below that the step is limited by the weight read and the GPU is wasted, so stay at ~500 or more.

Step time is the **max** of the two, not their sum, because the weight read and the math overlap.

> **RE-DRILL**
>
> Redo it for a 10 ms budget: 625 − 32 ≈ 593, so 512.